软件需求规格说明书

——抖音短视频平台（基于面向对象分析方法）

吴昌庆

2026 年 4 月 6 日

目录

1 引言 **3**
1.1 编写目的 3
1.2 系统范围 3
1.3 读者对象 3
1.4 术语与缩写解释 3
1.5 参考资料 3

2 面向对象分析：用例模型 **3**
2.1 系统参与者（Actors） 4
2.2 核心用例概述 4
2.3 用例规约：浏览视频流 4
2.4 用例规约：发布视频 4
2.5 用例间的关联关系 4

3 面向对象分析：对象类/领域模型 **4**
3.1 核心实体类定义 (Entity Classes) 5
3.2 辅助与交互类定义 5
3.3 控制类与边界类 (Control & Boundary Classes) 5
3.4 类间关系描述 5

4 面向对象分析：动态模型 **5**
4.1 发布视频顺序图描述 (Sequence Diagram) 6
4.2 视频对象状态转换流 (State Machine) 6
4.3 点赞交互活动描述 (Activity Flow) 6

4.4 用户鉴权协作说明 (Collaboration) 6

5 非功能性需求 **6**

5.1 性能需求 6

5.2 可用性与体验需求 7

5.3 安全性需求 7

5.4 可维护性与扩展性 7

5.5 运行环境需求 7

1 引言

1.1 编写目的

本文档旨在运用面向对象分析（OOA）方法，对“抖音”短视频平台进行全面的软件需求分析。通过构建用例模型、对象类模型及动态模型，清晰地定义系统的功能需求与非功能需求，为后续的面向对象设计（OOD）和系统开发提供基础标准。

1.2 系统范围

本系统为一款移动端为主的短视频社交平台，核心业务边界包括：用户账户管理、音视频内容发布与分发、用户社交互动（点赞、评论、关注、分享）以及基于算法的个性化内容推荐。

1.3 读者对象

本文档的主要读者包括：项目经理、系统架构师、前后端开发工程师、软件测试工程师以及 UI/UX 设计师。

1.4 术语与缩写解释

- **OOA (Object-Oriented Analysis)**：面向对象分析。
- **UGC (User Generated Content)**：用户生成内容，平台的核心内容来源。
- **Feed 流**：持续更新并呈现给用户的内容信息流。

1.5 参考资料

1. IEEE Std 830-1998《IEEE 软件需求规格说明建议实践》
2. 《系统分析与设计（面向对象方法）》
3. 《计算机软件需求规格说明规范》

2 面向对象分析：用例模型

用例模型描述了系统与外部参与者（Actor）之间的交互，定义了系统的功能边界。

2.1 系统参与者（Actors）

- **普通用户 (User)**：包括内容消费者（观看者）和内容创作者（发布者）。
- **管理员 (Administrator)**：负责内容审核、用户管理和系统维护。
- **推荐算法系统 (Recommendation Engine)**：外部子系统，负责根据用户行为画像推送视频。

2.2 核心用例概述

核心用例包括：浏览视频流、发布短视频、进行社交互动（点赞/评论/转发）、管理个人中心（关注/私信/资料修改）、以及内容安全审核。

2.3 用例规约：浏览视频流

- **前置条件**：用户已打开 App，网络连接正常。
- **基本流**：1. 系统请求推荐接口；2. 推荐引擎返回视频列表；3. 客户端自动预加载并播放第一个视频；4. 用户向上滑动，播放下一个视频。
- **异常流**：若网络断开，系统提示“网络不给力”，并尝试加载本地缓存视频。

2.4 用例规约：发布视频

- **前置条件**：用户已登录平台。
- **基本流**：1. 用户点击“+”号进入拍摄页面；2. 录制或从相册选择视频；3. 进入编辑界面添加滤镜/音乐/贴纸；4. 填写文案并设置可见范围；5. 点击发布，视频进入后台上传队列。

2.5 用例间的关联关系

- **包含关系 (Include)**：“发布视频”、“评论视频”用例均包含“用户身份验证”用例。
- **扩展关系 (Extend)**：在“浏览视频流”用例中，可扩展出“点击广告跳转”用例。

3 面向对象分析：对象类/领域模型

在领域驱动设计中，我们识别出以下核心业务实体及其属性与方法。

3.1 核心实体类定义 (Entity Classes)

1. 用户类 (User)

属性	`userID`, `username`, `passwordHash`, `avatar`, `followerCount`
方法	`register()`, `login()`, `follow(userID)`

2. 视频类 (Video)

属性	`videoID`, `authorID`, `videoURL`, `description`, `status`
方法	`play()`, `updateMetadata()`, `delete()`

3.2 辅助与交互类定义

互动类 (Interaction)：抽象关联类。

属性	`interactionID`, `userID`, `targetID`, `timestamp`, `type`
方法	`create()`, `revoke()`

3.3 控制类与边界类 (Control & Boundary Classes)

- **边界类**：`VideoFeedUI` (视频流展示界面), `CameraInterface` (拍摄界面)。
- **控制类**：`UploadController` (协调视频文件切片、上传服务器、保存数据库的控制逻辑)。

3.4 类间关系描述

- **聚合 (Aggregation)**：一个 `User` 对象聚合多个 `Video` 对象（用户的作品列表）。
- **继承 (Inheritance)**：`Like` (点赞类) 和 `Comment` (评论类) 继承自抽象基类 `Interaction`。

4 面向对象分析：动态模型

动态模型描述对象在特定场景下的状态变化和消息传递过程。

4.1 发布视频顺序图描述 (Sequence Diagram)

1. `User` 触发 `CameraInterface` 的 `startUpload()` 方法。
2. `UploadController` 实例化新的 `Video` 对象。
3. `Video` 对象向服务端发送数据块，状态置为 *Uploading*。
4. 上传完成后，系统调用内部 `AuditService.check()`，状态转为 *UnderReview*。
5. 审核通过，状态转为 *Published*。

4.2 视频对象状态转换流 (State Machine)

一个视频实体在系统中的生命周期状态包括：*Draft (草稿)* → *Uploading (上传中)* → *UnderReview (审核中)* → *Published (已发布)*。若审核违规，则转入 *Blocked (封禁)* 状态。

4.3 点赞交互活动描述 (Activity Flow)

1. 用户双击屏幕 → 2. 客户端向 `InteractionService` 发起请求 → 3. 校验用户登录状态 → 4. 写入点赞记录至数据库 → 5. `Video.likeCount` 累加 → 6. 客户端呈现红心动画特效。

4.4 用户鉴权协作说明 (Collaboration)

当用户尝试查看私信时，`MessageUI` 发送消息给 `AuthManager` 验证 Token 的有效性，若失效则拉起 `LoginUI`，在此过程中各对象协同保证系统安全性。

5 非功能性需求

为保障系统的可用性与健壮性，特定义以下非功能性约束。

5.1 性能需求

- **响应时间**：视频首帧加载时间需 ≤ 0.5 秒；API 接口响应时间需 ≤ 200 毫秒。
- **并发处理**：系统架构需支持千万级日活用户（DAU），并在晚高峰支持高并发的点赞与评论写入。

5.2 可用性与体验需求

采用极简的上下滑动（Swipe）交互逻辑，降低学习成本；核心播放页面应隐藏多余按钮，沉浸式展示视频内容；支持弱网环境下的降级策略（如降低清晰度以保证流畅播放）。

5.3 安全性需求

- 密码等敏感信息必须加盐进行 Hash 不可逆加密（如 bcrypt 算法）。
- 客户端与服务器的通信必须通过 HTTPS 加密，防止中间人攻击。
- 系统需配备防止恶意刷播放量、刷赞的防作弊策略。

5.4 可维护性与扩展性

后端架构应采用微服务架构（Microservices），将用户服务、视频服务、推荐服务解耦，以便单独扩容；数据库设计需支持分库分表，应对海量 UGC 数据的增长。

5.5 运行环境需求

- **客户端**：支持 iOS 12.0 及以上版本、Android 8.0 及以上版本。
- **服务端**：部署于 Linux (CentOS/Ubuntu) 环境，使用分布式云存储（如 AWS S3 或阿里云 OSS）存储音视频大文件。